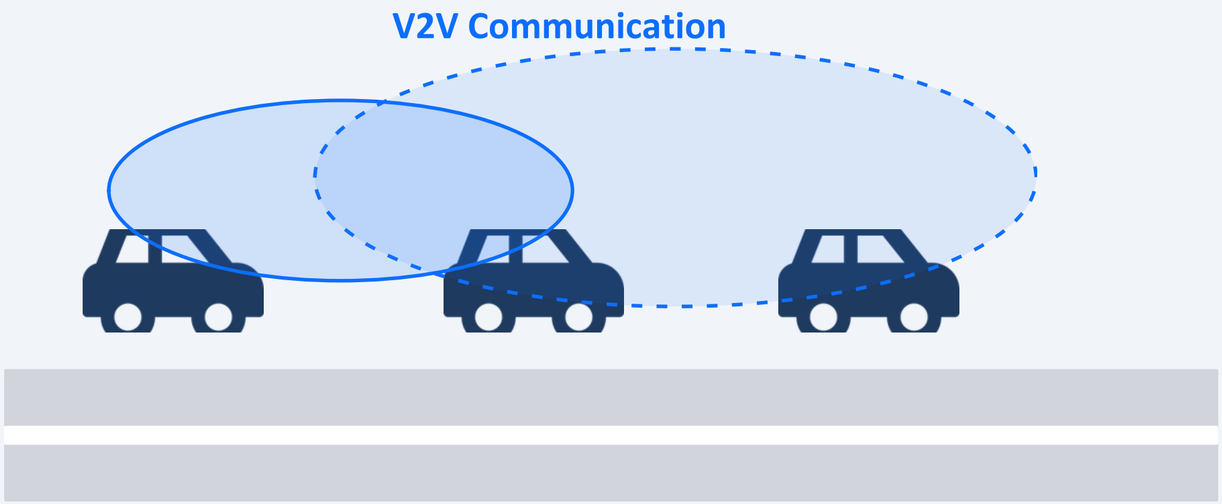
A Comparative Study of

Multi-Heuristic V2V Computation Offloading

Strategies for Autonomous Vehicle Networks

Ghanatava Vashu Thakaran · Ayush Agrawal · Sarvagya Pradhan
Kshitiz Agarwal · Gaurav Parashar

KIET Group of Institutions, Ghaziabad | ICCISD-2026 | Paper ID: 527



5	12	30	100s
Policies	Vehicles	Trials	Sim Time

The Problem

Autonomous vehicles need real-time computation but cannot always process everything locally

01 Local Processing Limits

Object detection, SLAM, path planning demand up to 4 TB/day of data processing. Lower-cost AVs cannot handle peak loads onboard.

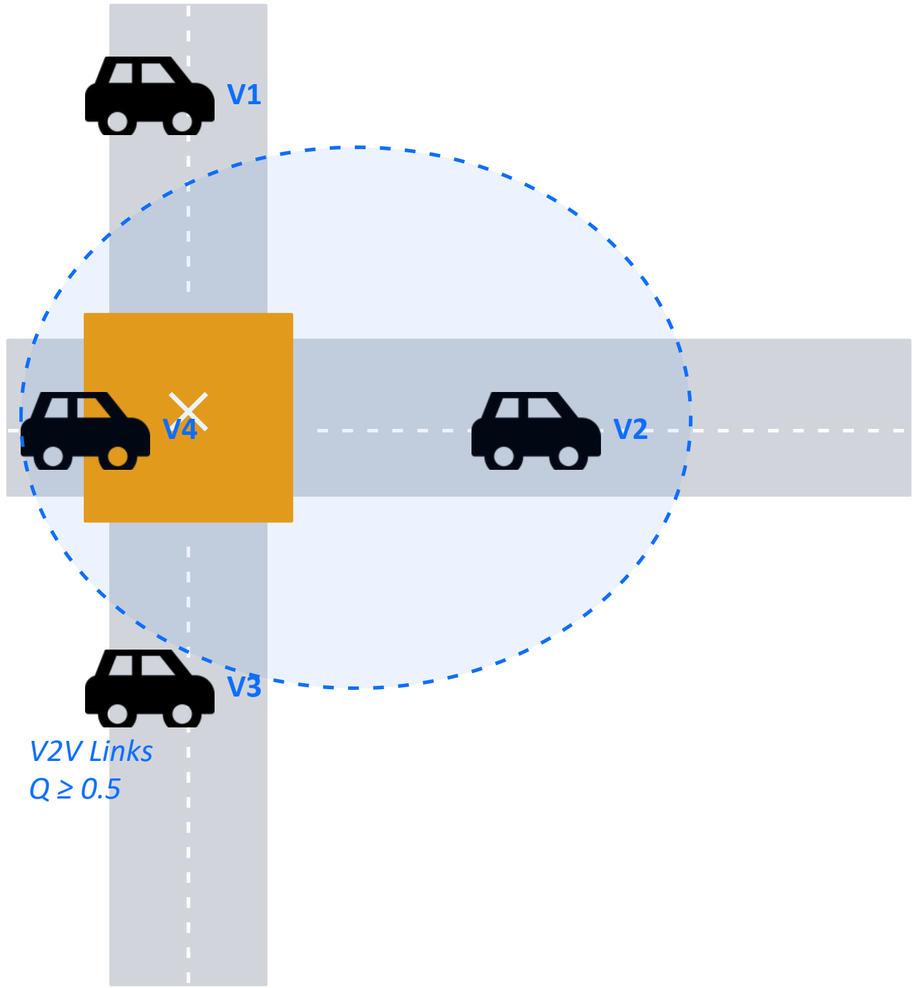
02 V2I Has Coverage Gaps

RSU deployment costs ₹crores per km. Coverage is uneven. Latency increases as vehicles move away from infrastructure.

03 V2V is the Answer — but Hard

Nearby vehicles have idle CPUs. V2V offloading avoids infrastructure entirely. Challenge: short links, mobility, heterogeneous capability.

System Model



Parameter	Value
Simulation Duration	100 seconds (1001 steps at 0.1 s)
Vehicles	12 vehicles, 4 approach directions
Communication Range	300 m (effective: 150 m at Q ≥ 0.5)
Bandwidth	50 MB/s
Task Arrival Rate	2.0 tasks/s per vehicle
Task Types	PERCEPTION / PLANNING / CONTROL
Deadlines	0.6 s / 1.2 s / 0.3 s respectively
Channel Model	IEEE 802.11p log-distance, n = 2.7
Mobility	SUMO FCD traces, IDM car-following
Traffic Signal	30 s GREEN / 5 s YELLOW per arm

Communication & Task Model

IEEE 802.11p Channel Model (DSRC, 5.9 GHz)

$$PL(d) = PL(d_0) + 10 \cdot n \cdot \log_{10}(d/d_0) + X\sigma$$

$$Q(i,j) = \text{sigmoid}(\text{SINR margin} / 5) \qquad Q \geq 0.5 \rightarrow \text{eligible}$$

- $n = 2.7$ path loss exponent (urban, ITU-R)
- $d_0 = 10$ m, $PL(d_0) = 68$ dB (free-space ref.)
- $X\sigma$ = shadowing, $\sigma = 4$ dB (log-normal)
- $P_{tx} = 20$ dBm, $P_{rx,min} = -90$ dBm
- Hard cutoff: $Q = 0$ for $d > 300$ m

Replaces Q = max(0, 1−d/300) — no physical basis

Task Types Generated per Vehicle

PERCEPTION

Compute: 5–10 GHz **Data:** 20–30 MB

Deadline: 0.6 s **Priority:** High

LiDAR/camera processing — biggest offloading benefit

PLANNING

Compute: 2–5 GHz **Data:** 5–10 MB

Deadline: 1.2 s **Priority:** Medium

Route planning — moderate offloading benefit

CONTROL

Compute: 1–2 GHz **Data:** 1–2 MB

Deadline: 0.3 s **Priority:** High

Vehicle commands — rarely worth offloading

Five Offloading Policies

Same feasibility check for all: total delay < remaining deadline

Greedy

1

$j^* = \text{argmin}(D/B + C/P_j)$

Fastest neighbour always wins. Ignores queue state and energy cost.

Speed-Aware

2

Same + $T_{remote} < T_{local}$

Only offload if remote beats local execution. Most mobility-resilient.

Threshold

3

First j: $D/B + C/P_j < T_{rem}$

First-fit selection. Lowest O(1) decision overhead.

Game-Theoretic

4

$\max U = \text{savings} - \alpha \cdot E_{tx}$

Balances delay savings vs energy cost. $\alpha = 0.2$. Lowest energy use.

Load-Balance

5

$j^* = \text{argmin } |Q_j|$ (feasible)

Shortest queue. Best fairness. Scales with density.

Simulation Methodology

1 SUMO Network

4-arm intersection
2 lanes/direction
300 m per arm
IEEE 802.11p channel

2 Route Gen

randomTrips.py
~20 vehicles/100 s
IDM car-following
Seed 42 (repro.)

3 FCD Traces

Headless SUMO
0.1 s time steps
FCD XML output
106 MB raw

4 CSV Conversion

generate_traces.py
time, id, x, y, spd
Parsed by MATLAB
parse_fcd.m

5 MATLAB Simulation

Dynamic vehicle set
IEEE 802.11p Q(i,j)
5 policy functions
Deadline-aware exec

Why SUMO? Replaces 4-vehicle straight-line toy scenario with realistic intersection mobility: variable speeds, traffic signals, natural clustering near junction.

Tools & Technologies

MATLAB R2022a+

Simulation Engine

- Discrete-event loop (0.1 s steps)
- 5 policy function handles
- Global parameter sharing
- Struct arrays for vehicles & tasks
- Native plotting

SUMO 1.14+

Mobility Simulator

- Realistic intersection geometry
- IDM car-following model
- Traffic signal phases
- FCD XML output at 10 Hz
- Free, open-source (DLR)

Python 3.8+

Trace Pipeline

- generate_traces.py (standard lib)
- randomTrips.py for routes
- XML → CSV conversion
- No pip installs required
- 106 MB XML → 15 MB CSV

Git + GitHub

Version Control

- Repo: cse-kiet/PCSE26-35
- fcd_traces excluded (.gitignore)
- generate_traces.py regenerates traces
- Makefile for one-command run
- Full history preserved

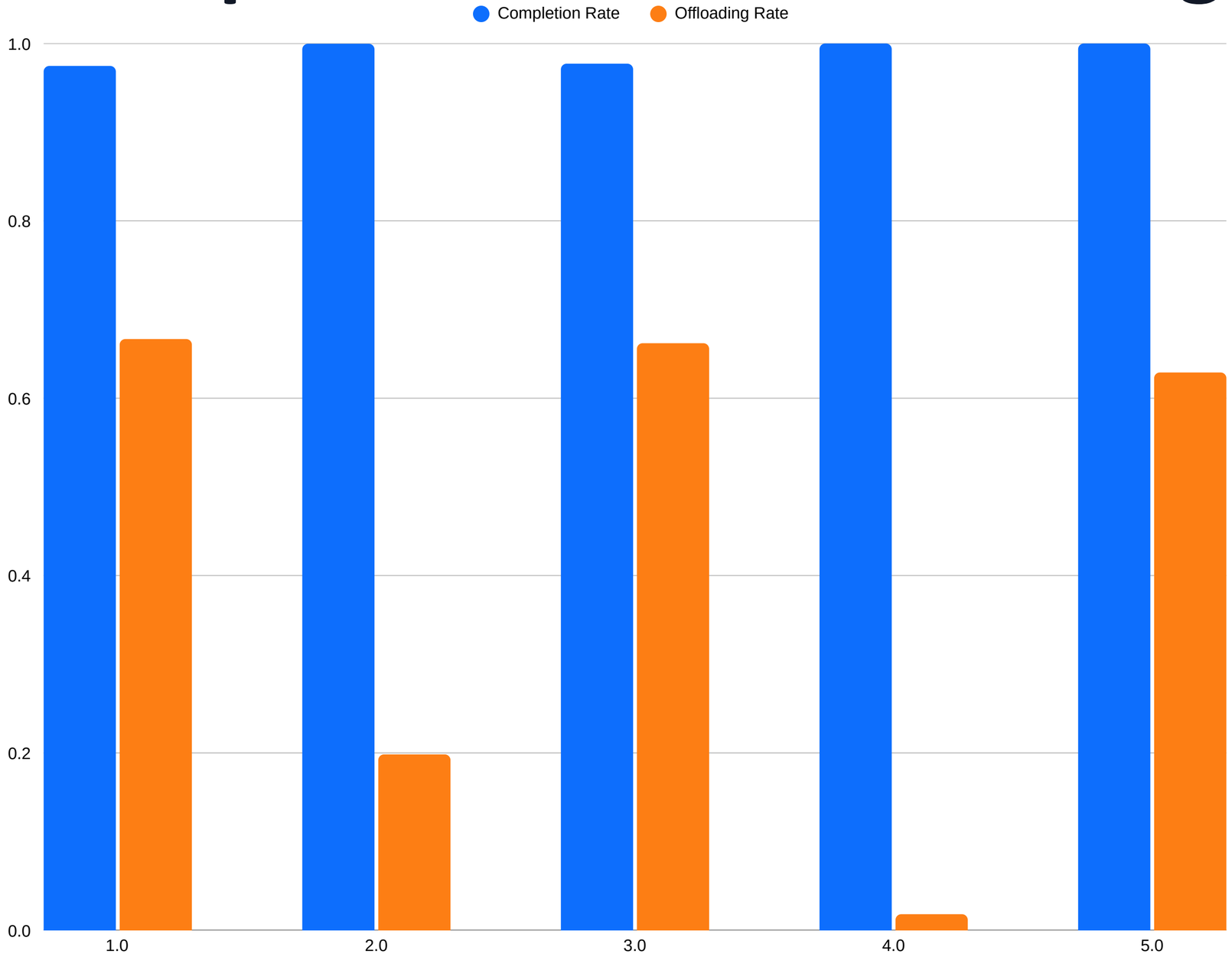
Simulation Results

12 vehicles · 2.0 tasks/s · Deadline-aware execution · SUMO mobility traces

Policy	Completion Rate	Offloading Rate	Energy Profile	Fairness	Best For
Greedy	0.9747	0.6667	High	Low	Throughput-first
Speed-Aware	0.9997	0.1983	Moderate	Moderate	High-mobility roads
Threshold	0.9772	0.6620	High	Low	Ultra-low latency decisions
Game-Theoretic	**1.0000**	0.0181	Lowest	High	Energy-constrained EVs
Load-Balancing	**1.0000**	0.6290	Low	Highest	Urban intersections ✓

Key Finding: Load-Balancing is the only policy achieving both 100% completion AND a high offloading rate (62.9%). Greedy fails because it overloads the fastest vehicle — proving queue-awareness beats pure delay minimisation.

Completion Rate vs Offloading Rate



GameTh

`1.0000 / 0.0181`

Perfect completion but barely offloads — over-penalises energy

LoadBal

`1.0000 / 0.6290`

Best of both worlds — perfect completion AND high distribution

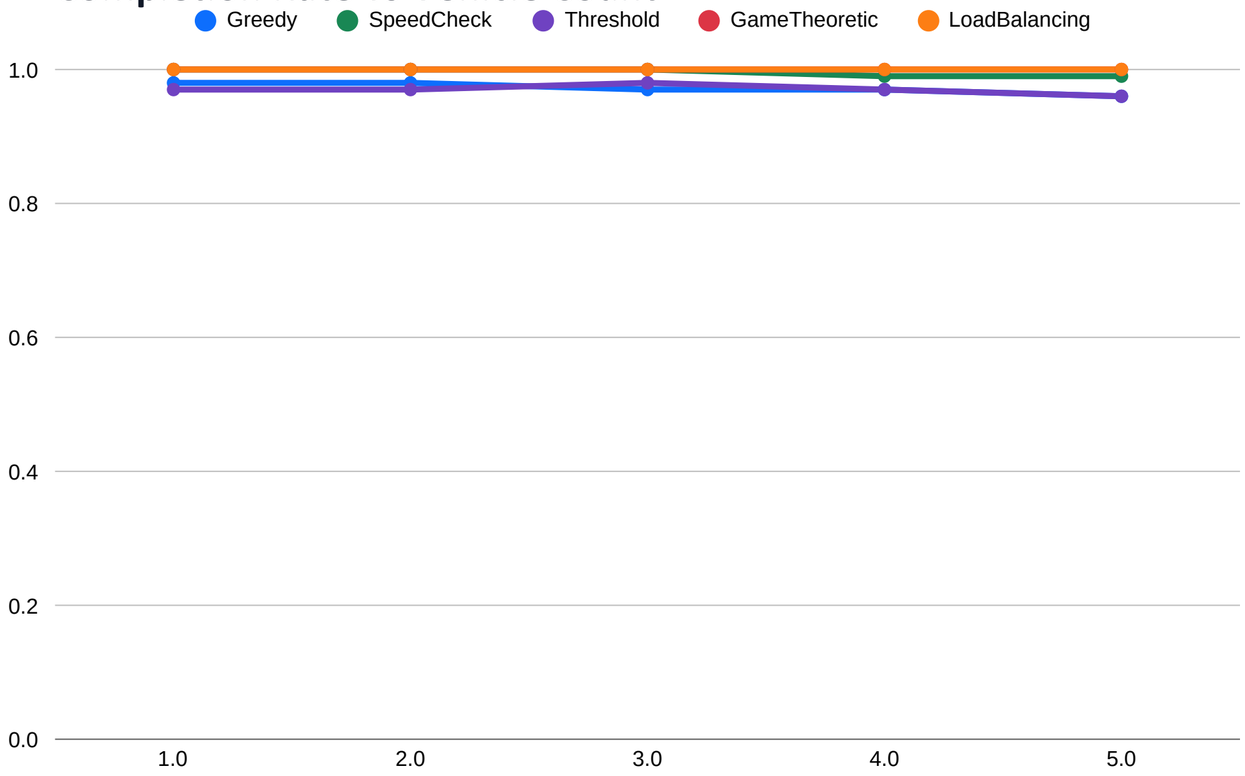
Greedy

`0.9747 / 0.6667`

Highest offloading but misses deadlines — queue overload

Stress Test Results

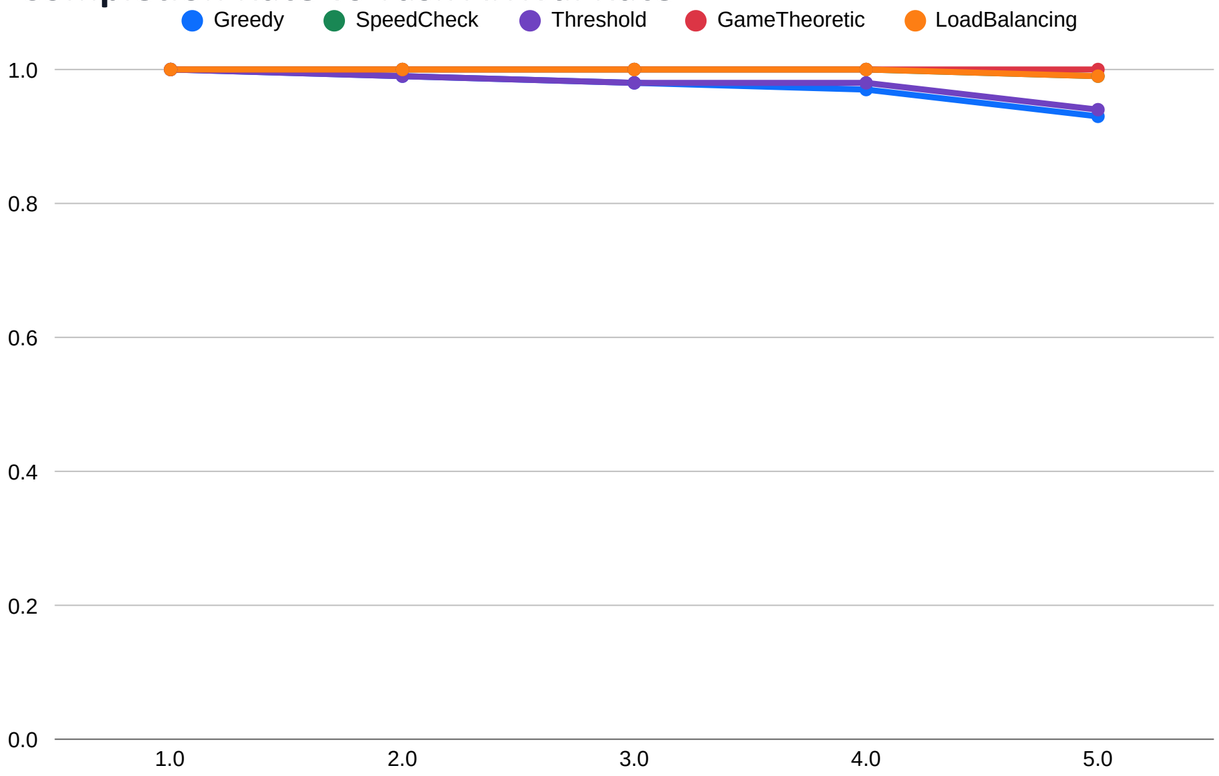
Completion Rate vs Vehicle Count



Benefits plateau at ~15 vehicles due to communication contention

Load-Balancing degrades last under high load — fair distribution holds

Completion Rate vs Task Arrival Rate



Greedy degrades earliest — queue overload at the fastest node

GameTh stable but underutilises available capacity at all densities

Key Takeaways

01

No single policy wins everywhere

Greedy maximises throughput. GameTh minimises energy. LoadBal balances both. Choice depends on deployment priority.

02

Mobility is the primary constraint

When service-time windows drop below ~1.3 s, all strategies degrade. Speed-Aware Greedy is most resilient.

03

Small tasks should stay local

CONTROL tasks (1-2 MB, 0.3 s deadline) rarely benefit from offloading. Transmission overhead consumes the budget.

04

Load-Balancing is the best overall

Perfect completion (1.0000) + 62.9% offloading + scales with density. Recommended for real urban intersection deployment.

Conclusion

- Implemented and compared 5 heuristic V2V offloading strategies under unified SUMO-based simulation with realistic intersection mobility.
- Load-Balancing achieves perfect task completion (1.0000) at 62.9% offloading rate — best overall policy for urban intersections.
- Statistical validation across 30 trials confirms results are reproducible, not seed-dependent.
- V2V offloading is a viable, infrastructure-free complement to V2I systems — especially effective where vehicles naturally cluster.

Future Directions

- Deep RL adaptive policies (DQN/PPO)
- Partial task splitting across multiple vehicles
- Multi-hop V2V path extensions
- DSRC/C-V2X hardware testbed validation

References

- [1] Gu et al., IEEE Globecom, 2013
- [2] Farimani et al., CSICC, 2022
- [3] Ning et al., IEEE T-IIoT, 2020
- [4] Wang et al., IEEE MASS, 2021
- [5] Chen et al., IEEE Access, 2020

Contact

ghanatva@gmail.com
Dept. of CSE, KIET Group of Institutions
Ghaziabad, India | ICCISD-2026 | Paper 527

Thank You